03

Part 3 · Storage & Databases

Where data lives in the cloud and how it is kept safe, durable and recoverable.

- Object · Block · File storage
- Buckets
- Durability vs Availability
- Backup vs Replication
- Managed databases

3.1 Object Storage

★★★

IN ONE LINE

Flat, scalable storage of data as objects (data + metadata + key) accessed through an HTTP API.

Object storage stores data as **objects** — the data itself, its metadata and a unique key — in a flat namespace, accessed over HTTP APIs. It is almost infinitely scalable, very cheap and extremely durable. Examples: **Amazon S3, Azure Blob Storage, Google Cloud Storage**.

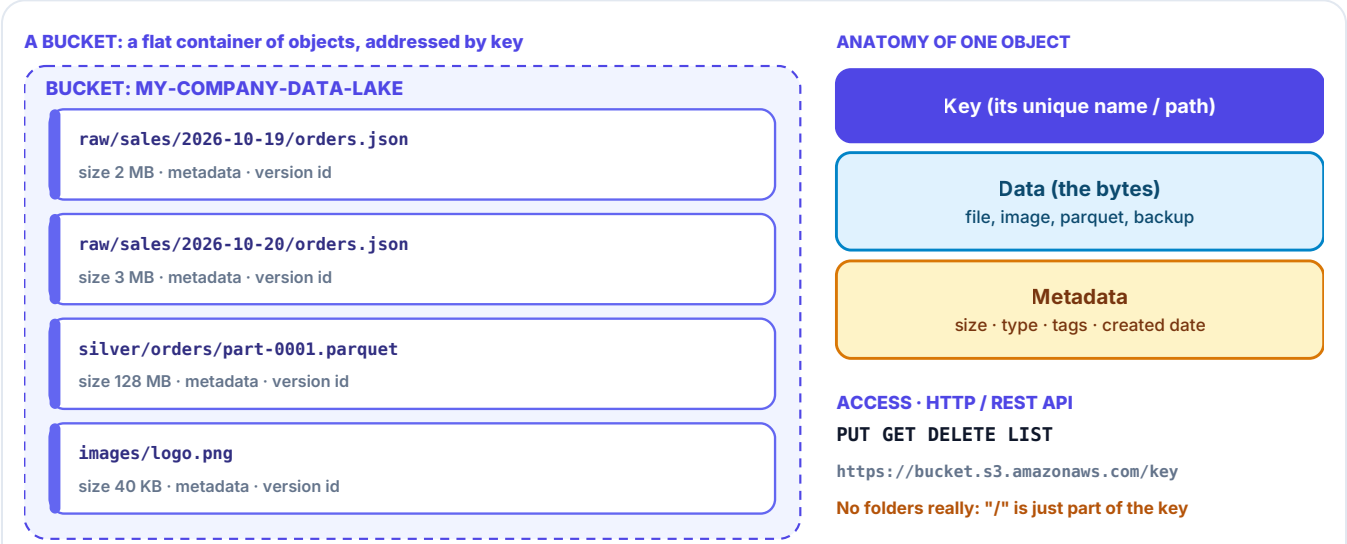


Figure 3.1 — Objects live in buckets and are addressed by key

Great for

- Backups and archives
- Images, video, documents
- **Data lakes** (Parquet, JSON, CSV)
- Static website hosting
- Logs and ML training data

Not great for

- Operating system disks
- Databases needing small random writes
- Frequently **editing part of a file** (objects are replaced whole)
- Low-latency transactional workloads

COMMON TRAP

Objects are **immutable**: to change a byte you re-upload the whole object. That is exactly why lake file formats like Parquet plus table formats (Delta, Iceberg) exist.

DATA ENGINEERING TIE-IN

Object storage is the **foundation of the data lake and lakehouse**. Bronze/Silver/Gold layers are just prefixes in a bucket. Know: S3 = IaaS storage; cheap; decoupled from compute; use partitioned Parquet, avoid millions of tiny files.

3.2 Block Storage

★★★

IN ONE LINE

Raw, fixed-size blocks presented to a server as a fast disk; ideal for OS disks and databases.

Block storage splits data into equal-sized **blocks** and presents them to a server as a raw disk. The OS puts a file system on it, so it behaves like the C: drive or `/dev/sda`. It gives **low latency and high IOPS**, which is why databases and boot disks use it. Examples: **Amazon EBS, Azure Managed Disks, Google Persistent Disk**.

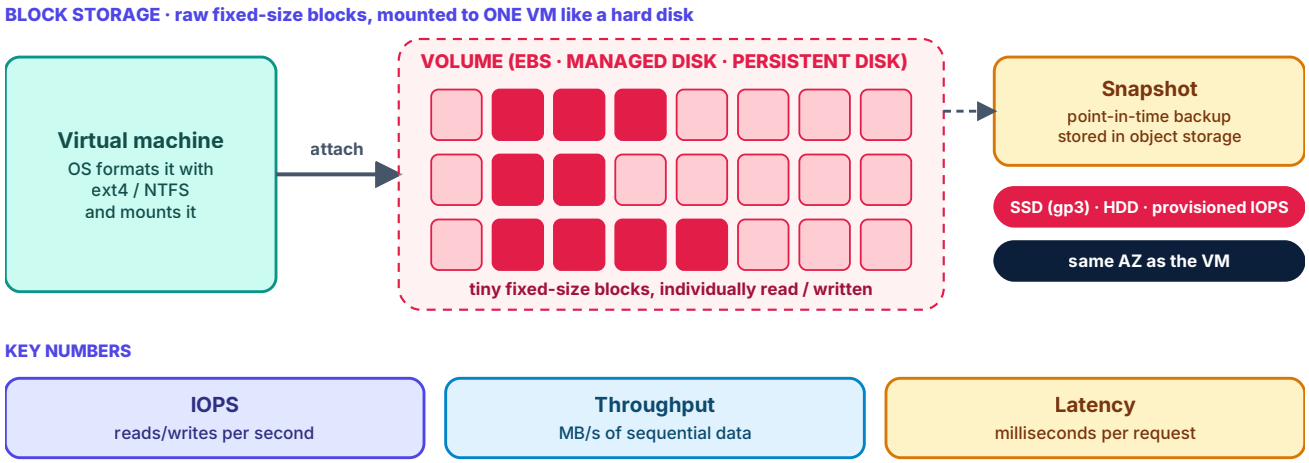


Figure 3.2 — A block volume attached to a VM, with snapshots

Characteristics

- Attached to **one** VM at a time (usually) and in the **same Availability Zone**.
- Persists independently of the VM (data survives a stop).
- Resizable; choose SSD for speed or HDD for cheap throughput.
- Back up with **snapshots**.

Typical uses

- VM boot / OS disks
- Relational database storage (MySQL, PostgreSQL)
- Transaction-heavy and low-latency apps
- Self-managed Kafka / Elasticsearch nodes

COMMON TRAP

Instance store (disks physically inside the host) is *ephemeral*: data is lost when the VM stops or fails. Block volumes (EBS) persist. Never keep the only copy of important data on ephemeral storage.

3.3 File Storage

★★☆

IN ONE LINE

A shared, hierarchical network file system (NFS / SMB) mounted by many servers at once.

File storage organises data in the familiar **hierarchy of folders and files** and lets **many machines share it at once** over a network protocol (**NFS** on Linux, **SMB/CIFS** on Windows). It is a managed "network drive". Examples: **Amazon EFS / FSx, Azure Files, Google Filestore**.

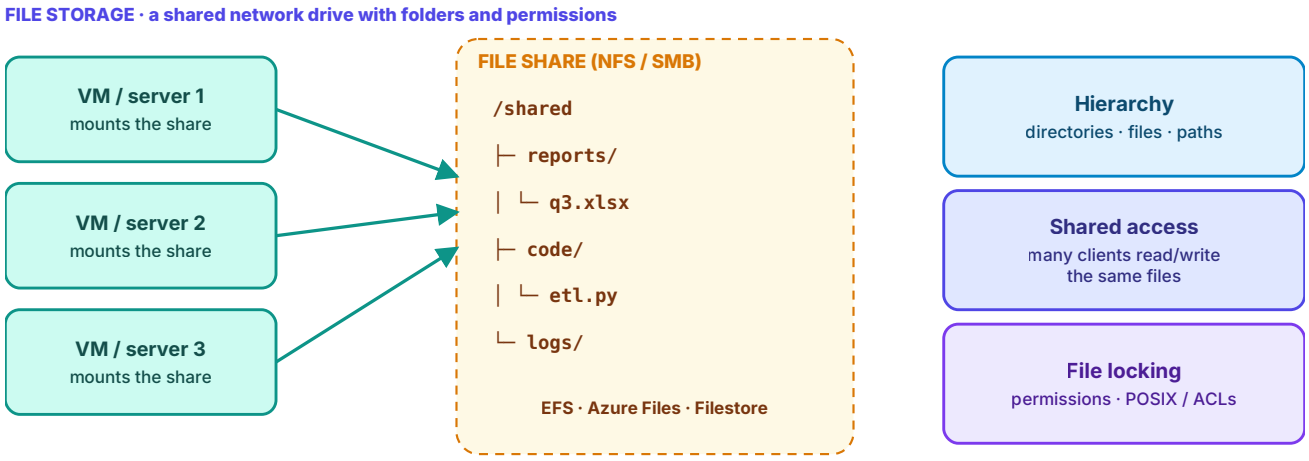


Figure 3.3 — Several servers mounting one shared file system

Typical uses

- Shared content for several web servers
- Home directories, team shares
- Lift-and-shift of apps that expect a file system
- Dev tools, CI build caches

Limits

- More expensive per GB than object storage
- Slower than block storage for databases
- Scaling limits vs object storage
- Needs network mounting (NFS / SMB)

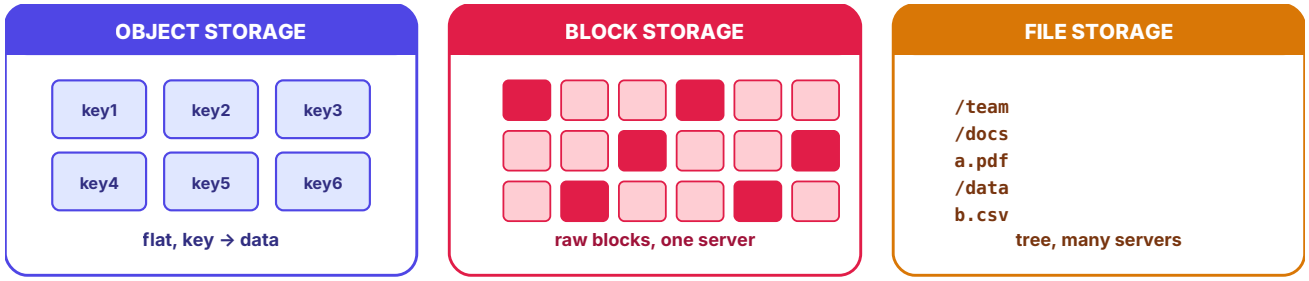
3.4 Object vs Block vs File Storage

★★★

IN ONE LINE

Choosing the right storage type by structure, access pattern, speed and cost.

The three storage types suit different jobs. A good fresher answer picks by **access pattern**: raw speed for one server (block), shared folders (file), or massive cheap scalable storage (object).



	Object	Block	File
Access	HTTP API (REST)	Mounted as a disk	NFS / SMB mount
Latency	Higher	Lowest	Medium
Scale	Virtually unlimited	TBs per volume	Large, shared
Cost per GB	Lowest	Highest	Medium

Figure 3.4a — Side-by-side comparison

WHICH ONE SHOULD I USE?

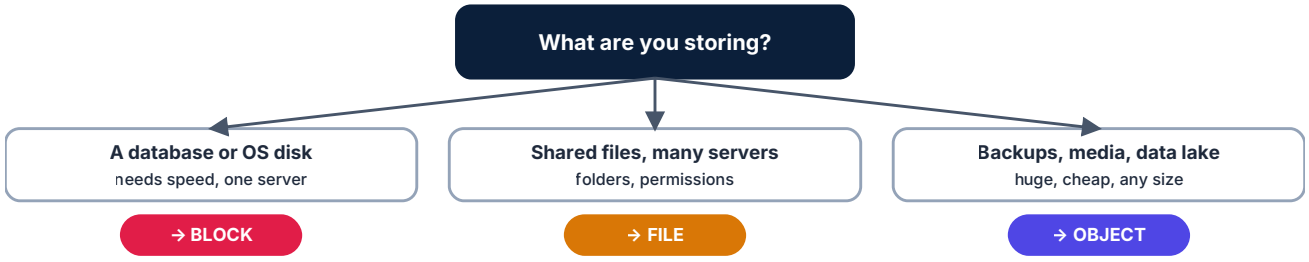


Figure 3.4b — A quick decision guide

	Object	Block	File
Structure	Flat objects with metadata	Fixed-size blocks	Hierarchical files and folders
Shared by many servers?	Yes (HTTP)	Usually one at a time	Yes (NFS / SMB)
Modify part of data?	No, replace whole object	Yes, per block	Yes, per file
Best for	Data lakes, backups, media, static sites	Databases, boot volumes	Shared content, home directories
AWS / Azure / GCP	S3 / Blob / Cloud Storage	EBS / Managed Disks / Persistent Disk	EFS / Azure Files / Filestore

REAL-WORLD ANALOGY

Object = a huge warehouse where every box has a label and you ask for it by label. Block = the hard drive inside your own computer. File = a shared office filing cabinet that everyone opens at once.

3.5 Storage Bucket

★★★

IN ONE LINE

The globally named, region-bound container that holds objects and carries access and lifecycle settings.

A **storage bucket** is the top-level container in object storage. You create a bucket, then upload objects into it. The bucket name is **globally unique**, belongs to a **region**, and is where you set access rules, encryption, versioning and lifecycle policies.

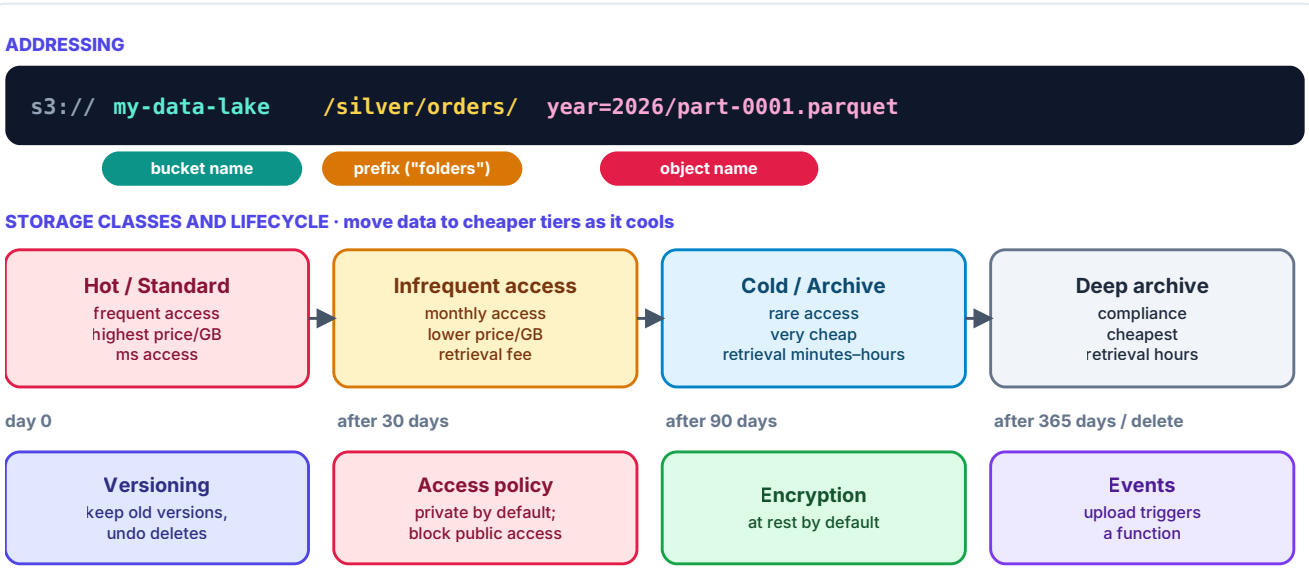


Figure 3.5 — Bucket addressing, storage tiers and key features

Feature	What it does
Region	Data physically stays in the chosen region (important for latency and data-residency laws).
Access control	Bucket policies + IAM + ACLs. **Default private.** Turn on “block public access” unless you are deliberately hosting a public site.
Versioning	Keeps every version of an object so you can recover from overwrites and deletes.
Lifecycle rules	Automatically transition objects to cheaper tiers or delete them after N days.
Replication	Copy objects to another bucket or region for DR or lower read latency.
Event notifications	Trigger functions or queues on upload, e.g. start an ETL job.

COMMON TRAP

Public buckets are the #1 cloud data leak. A misconfigured bucket policy exposes data to the whole internet. Keep buckets private, give access through IAM roles and use short-lived signed URLs for sharing.

DATA ENGINEERING TIE-IN

Lakehouse layout: `s3://lake/bronze/` , `/silver/` , `/gold/` . Use lifecycle rules to age raw Bronze data into cheaper tiers, and partition prefixes (e.g. `year=2026/month=10/`) so engines can prune.

3.6 Data Durability vs Data Availability

★★★

IN ONE LINE

Durability: data is not lost. Availability: data is reachable right now.

Two different promises that are often confused. **Durability** is the probability that stored data is **not lost or corrupted** over time. **Availability** is the percentage of time a service is **up and responding**.

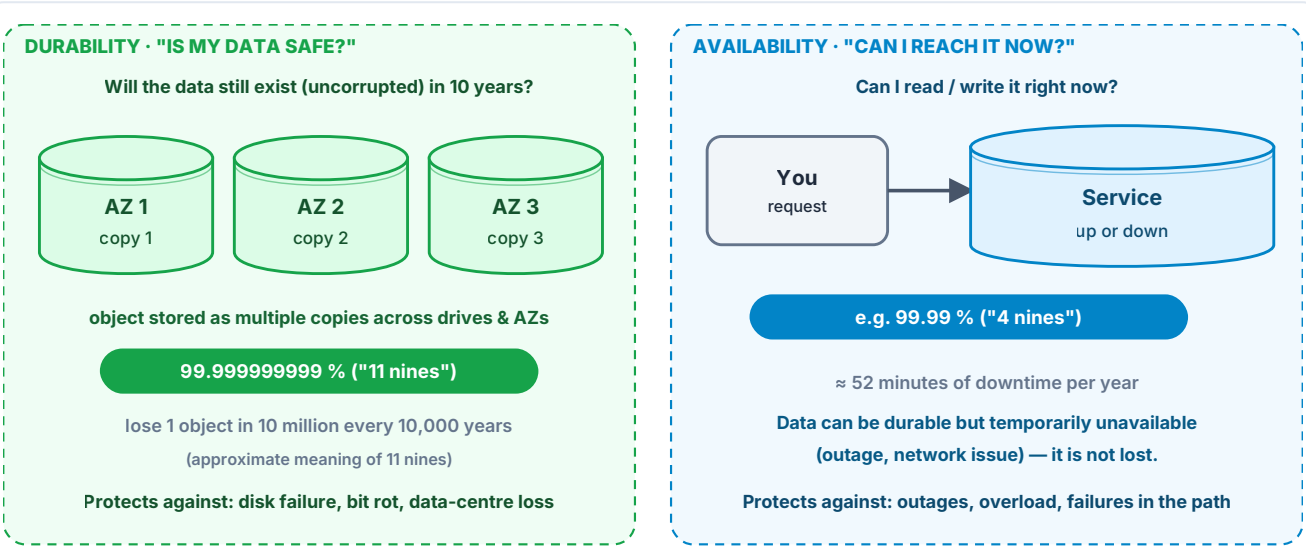


Figure 3.6 — Safe data vs reachable data

Availability	Downtime per year	Per month	Common name
99 %	3.65 days	7.3 hours	two nines
99.9 %	8.76 hours	43.8 minutes	three nines
99.95 %	4.38 hours	21.9 minutes	
99.99 %	52.6 minutes	4.4 minutes	four nines
99.999 %	5.26 minutes	26 seconds	five nines

How durability is achieved

- Store **multiple copies** across disks and Availability Zones.
- Checksums detect and repair corruption.
- Erasure coding and background healing.
- Versioning protects against human error.

How availability is achieved

- Redundant servers behind a load balancer.
- Deploy across multiple Availability Zones.
- Health checks and automatic failover.
- Graceful handling of overload.

INTERVIEW ANGLE

Memorise the line: **"Durability is about not losing the data; availability is about being able to access it."** Object storage typically advertises ~11 nines of durability but a lower figure (e.g. 99.9–99.99 %) for availability. Durability does *not* protect against you deleting data, which is why you also need backups and versioning (next topic).

3.7 Backup vs Replication

★★★

IN ONE LINE

Replication keeps a live copy for availability; backups keep point-in-time copies for recovery.

Replication continuously copies data to another location so a failure does not stop you — it is about **availability**. **Backup** takes **point-in-time copies** you can go back to — it is about **recovery from mistakes and corruption**. You need both.

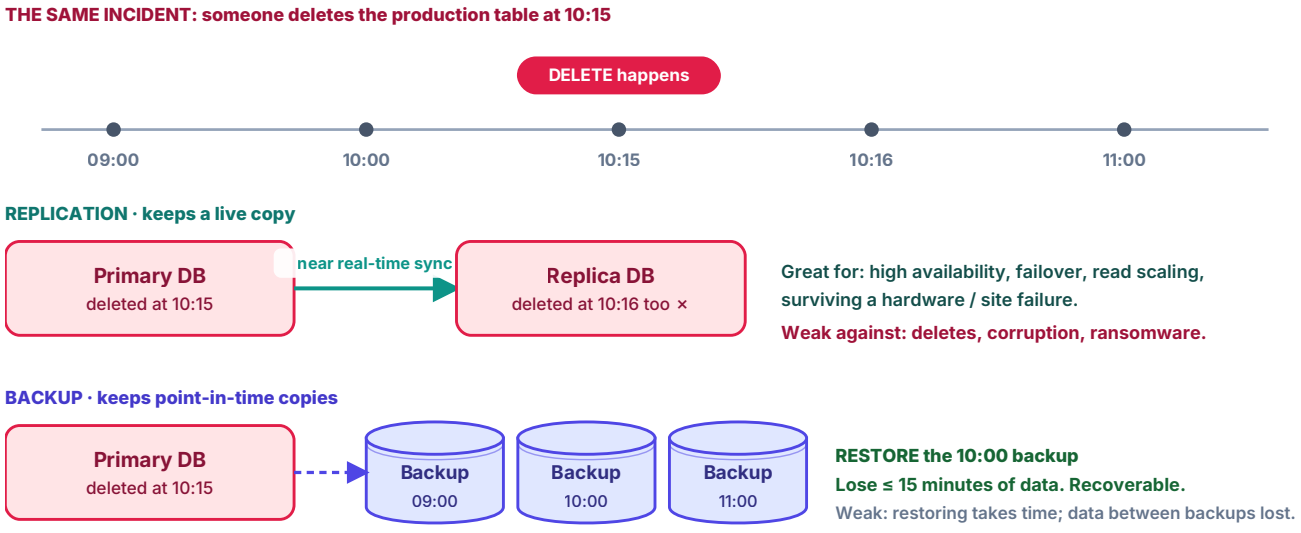


Figure 3.7 — A delete replicates instantly; a backup lets you go back in time

	Replication	Backup
Goal	High availability, fast failover, DR	Recover from deletion, corruption, ransomware, bad deploys
Freshness	Near real-time (sync / async)	As of the last backup (hourly, daily)
History	Only the latest state	Many point-in-time versions
Mistakes	**Replicates mistakes** instantly	Lets you roll back before the mistake
Recovery speed	Seconds to minutes (failover)	Minutes to hours (restore)
Cost	Higher (a second live system)	Lower (storage of snapshots)

The 3-2-1 backup rule

3 copies of your data, on **2** different types of storage, with **1** copy off-site (another region / account). Many add "immutable" backups so ransomware cannot delete them.

Replication flavours

- **Synchronous** — write confirmed on both; no data loss, higher latency.
- **Asynchronous** — replica lags slightly; faster, small risk of loss.
- Same-AZ, cross-AZ, **cross-region**.

INTERVIEW ANGLE

The classic interview question: *"Is replication a backup?"* **No.** If you drop a table, the replica drops it too. Always pair replication (for uptime) with backups or versioning (for recovery).

3.8 Managed Databases in the Cloud

★★☆

IN ONE LINE

Provider-run SQL and NoSQL databases: the provider operates them, you design and query.

Cloud providers sell **managed databases** (DBaaS): the provider handles installation, patching, backups, replication and failover; you handle schema and queries. It is a classic **PaaS** example and a favourite fresher topic because it links storage, availability and cost.

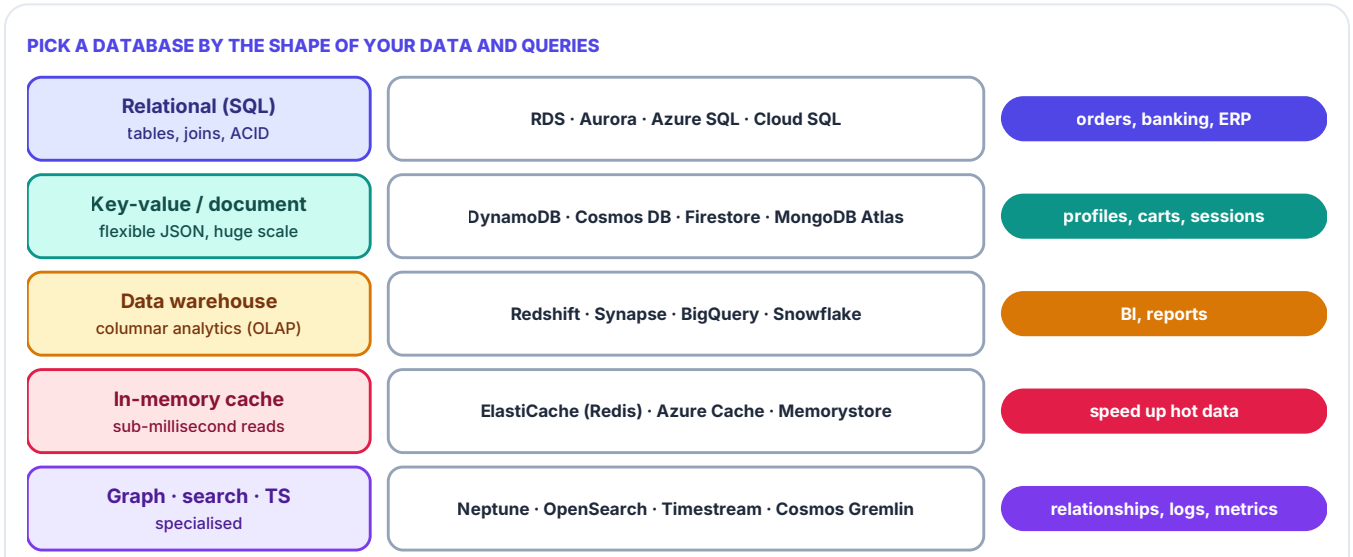


Figure 3.8 — Database families and cloud examples

	SQL (relational)	NoSQL
Schema	Fixed, defined up front	Flexible / schema-less
Scaling	Mostly vertical, then read replicas / sharding	Built for horizontal scale-out
Consistency	ACID transactions	Often eventual (tunable)
Query	SQL, joins	Key lookups, document queries
Use when	Relationships and correctness matter	Huge scale, simple access patterns, changing data shape

What "managed" gives you

- Automated backups and point-in-time restore
- **Multi-AZ** standby with automatic failover
- Read replicas for scaling reads
- Patching and monitoring
- Encryption at rest by default

Still your job

- Schema design and indexing
- Query performance
- Access control and network exposure
- Cost: instance size, storage, I/O

DATA ENGINEERING TIE-IN

OLTP databases (RDS/Aurora) serve the application; data is copied via **CDC or ETL** into an OLAP **warehouse or lakehouse** for analytics. Refer back to the Data Engineering handbook (OLTP vs OLAP, CDC, Lakebase).